

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

C03: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Code of Conduct:

IMOHAMED NAWAS (192521037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: MOHAMED NAWAS N

QUESTIONS:4**Total Marks:40**

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

aQuestions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS, DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

1. Examine the impact of centralized DNS architecture on application performance.
2. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
3. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4. A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with	Good analysis with logical	Basic analysis with limited	Weak or no analysis;

		strong reasoning and insights	reasoning	depth	lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: Video Conferencing Performance Problems

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer

The video conferencing problem is primarily associated with the **choice and behavior of the transport protocol, particularly UDP-based real-time communication, combined with application-level buffering and congestion-management mechanisms** . The reported network conditions are **8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization** , which are highly problematic for real-time multimedia communication.

Most modern video conferencing systems prefer **UDP** for transmitting audio and video because UDP does not require retransmission of every lost packet. TCP, in contrast, guarantees reliable and ordered delivery by retransmitting lost packets. Although this is beneficial for applications such as file transfer and web pages, retransmissions and congestion control can introduce significant delays in real-time communication.

The 8% packet loss can directly cause missing audio/video information. In an audio stream, packet loss may result in voice gaps or distortion. In video, it can produce frozen frames, visual artifacts, or temporary degradation. The **180 ms jitter** indicates considerable variation in packet arrival times. If the application's jitter buffer is too small, packets that arrive late may be discarded. If the buffer is too large, it introduces additional end-to-end delay.

Therefore, the issue cannot be attributed entirely to either the transport or application layer.

The **network and transport behavior create the adverse conditions** , while the **application's buffering, packet-loss concealment, adaptive bitrate, and congestion-control mechanisms determine how severely users experience them** .

2. Justify the answer based on the communication requirements of real-time multimedia applications

Real-time multimedia applications have requirements that differ considerably from traditional data-transfer applications. The most important characteristics are **low latency, low jitter, controlled packet loss, and continuous data delivery** .

For a file-transfer application, losing a packet is generally acceptable because the protocol can retransmit it. However, retransmitting an old audio or video packet may not be useful in a live conference. For example, if a video packet belonging to a frame displayed 500 milliseconds earlier is retransmitted, the information may arrive too late to be useful. Consequently, real-time applications generally prefer timely delivery over perfect reliability.

Importance of the observed network conditions

Network condition	Effect on video conferencing
8% packet loss	Missing audio/video packets, distortion and frozen frames
180 ms jitter	Irregular packet arrival and playback interruptions
Fluctuating bandwidth	Quality changes, buffering and bitrate adaptation
High retransmission	Additional delay if TCP is used
Excessive buffering	Increased latency and delayed conversations

UDP is therefore generally better suited to the basic transport requirements of interactive audio/video. However, UDP itself does not provide congestion control, reliability, or packet ordering. These functions must be implemented by the conferencing application or by protocols built above UDP.

Application-level mechanisms such as **jitter buffers, adaptive bitrate control, forward error correction (FEC), packet-loss concealment, and congestion control** are therefore essential.

The main objective should not simply be to eliminate every packet loss. Instead, the system

should ensure that audio and video packets arrive **quickly enough and regularly enough to maintain an acceptable user experience** .

3. Recommend suitable protocol-level or application-level improvements

Several improvements can be implemented.

a. Use UDP-based real-time transport

If the conferencing platform is currently using TCP for media streams, real-time audio and video should preferably be transported using UDP-based mechanisms. Technologies such as **WebRTC** commonly use UDP-based transport for interactive media.

b. Implement adaptive jitter buffering

The application should dynamically adjust its jitter buffer according to current network conditions. When jitter increases, the buffer can temporarily increase to accommodate delayed packets. When the network becomes stable, the buffer can be reduced to minimize latency.

c. Adaptive bitrate control

The video encoder should continuously monitor bandwidth, packet loss, and congestion.

When network capacity decreases, the system should automatically reduce:

- Video resolution

- Frame rate

- Bitrate

- Video quality

This is preferable to allowing the stream to freeze completely.

d. Forward Error Correction

Forward Error Correction (FEC) can add redundant information to transmitted media packets. The receiver can use this redundant information to reconstruct some lost packets without waiting for retransmission.

e. Packet-loss concealment

For audio, packet-loss concealment techniques can generate a reasonable approximation of missing audio samples. This can reduce the noticeable effect of packet loss.

f. Improve congestion control

The application should use congestion-control mechanisms designed for real-time traffic.

These mechanisms should respond quickly to bandwidth changes without causing excessive buffering.

g. Network Quality of Service

The organization can implement **QoS policies** to prioritize voice and video traffic over less time-sensitive traffic such as large downloads, software updates, or backups.

Conclusion

The problem is a combination of **transport/network conditions and application-level handling**, but the symptoms strongly indicate that real-time traffic is being affected by high packet loss and jitter. A UDP-based real-time transport mechanism combined with adaptive bitrate, intelligent jitter buffering, FEC, packet-loss concealment, and QoS can significantly improve conferencing quality.

Case Study 2: Redesigning DNS Infrastructure

1. Analyze the reasons behind the high DNS resolution time

An average DNS lookup time of **800 ms** is extremely high for a web application and can significantly increase the time required before a webpage begins loading.

DNS resolution normally involves translating a human-readable domain name into an IP address. Depending on caching and the DNS architecture, a query may involve multiple steps between the client, recursive resolver, root DNS servers, Top-Level Domain (TLD) servers, and authoritative DNS servers.

Several factors may contribute to the 800 ms delay.

a. Geographic distance

If users in Asia are querying a DNS server located in North America or Europe, network round-trip time can increase. A geographically distant resolver or authoritative server can therefore increase DNS latency.

b. Centralized DNS infrastructure

If DNS requests are concentrated on a small number of servers, the servers may become overloaded during peak periods. Processing queues and network congestion can increase response time.

c. Poor caching

If DNS responses are not cached effectively, the recursive resolver may need to repeatedly perform complete DNS resolution processes.

d. Incorrect TTL configuration

Very short TTL values cause cached records to expire frequently. This results in more DNS queries and increased load on authoritative DNS servers.

e. Network congestion

Packet loss and network congestion between clients, recursive resolvers, and authoritative servers can cause DNS queries to be retransmitted or delayed.

f. Slow or overloaded DNS servers

Insufficient server resources, high CPU usage, limited network capacity, or poor DNS software configuration can increase processing time.

g. Lack of redundancy

A single DNS server or a small number of geographically concentrated DNS servers creates both a performance bottleneck and a single point of failure.

2. Proposed DNS architecture using Anycast DNS, DNS pre-fetching, and TTL optimization

A suitable solution is a **globally distributed, multi-layer DNS architecture** .

Proposed architecture

Users → Local/ISP Recursive Resolver → Anycast DNS Infrastructure → Authoritative DNS

Servers

The architecture can contain the following components:

Layer 1: Local DNS caching

Employees and end users should use nearby recursive DNS resolvers. Frequently requested domain records can be stored in the resolver cache.

Layer 2: Anycast DNS

The organization should deploy DNS servers at multiple geographically distributed locations and advertise the same IP address using **Anycast routing** .

For example, DNS infrastructure could be deployed in:

North America

Europe

Asia

South America

Australia

When a user sends a DNS query, Internet routing directs the request toward an appropriate or nearby Anycast location.

This reduces network round-trip time and distributes DNS traffic across multiple sites.

Layer 3: DNS caching

Recursive DNS resolvers should cache frequently requested records. When a valid cached record exists, the resolver can answer without contacting the authoritative server.

Layer 4: DNS pre-fetching

DNS pre-fetching allows frequently accessed or soon-to-expire DNS records to be refreshed

before users request them. This reduces the probability that users experience delays because a cached DNS record has expired.

Layer 5: TTL optimization

TTL values should be selected according to how frequently DNS records change.

For stable services, a moderately long TTL can reduce repeated DNS lookups. For rapidly changing services, a shorter TTL may be necessary.

A reasonable starting strategy might be:

Static/stable records: **1–24 hours**

Frequently changed records: **5–60 minutes**

Critical infrastructure: TTL based on operational requirements

The exact values should be determined through monitoring and testing rather than applying one value to every record.

Target of below 50 ms

The proposed architecture can significantly reduce lookup latency by placing DNS infrastructure closer to users and increasing cache-hit rates. However, achieving **less than 50 ms for every DNS query worldwide cannot be guaranteed**, because DNS latency depends on the user's network, recursive resolver, Internet routing, packet loss, and other external factors.

The realistic objective should be to achieve **sub-50-ms latency for the large majority of cached or geographically close queries**, while continuously monitoring the remaining cases.

3. Advantages and limitations

Feature	Advantages	Limitations
Anycast DNS	Low latency, global distribution, fault tolerance	More complex routing and monitoring

DNS caching	Reduces repeated queries and latency	Cached information can become outdated
DNS pre-fetching	Reduces cache-miss delays	Generates additional background queries
TTL optimization	Balances freshness and performance	Incorrect values can increase load or cause stale records
Multiple DNS sites	High availability	Higher infrastructure and management cost

Performance

The architecture reduces DNS latency by shortening the network path and increasing cache utilization.

Scalability

Anycast and distributed authoritative servers allow DNS queries to be distributed across multiple locations rather than concentrating them on one server.

Availability

If one DNS site becomes unavailable, routing can direct traffic toward another operational site. This removes the single point of failure.

Conclusion

The best solution is a **distributed Anycast DNS architecture supported by recursive caching, DNS pre-fetching, and carefully optimized TTL values** . This architecture improves performance, scalability, and availability while reducing the probability of DNS becoming the bottleneck in webpage loading.

Case Study 3: Highly Available Global DNS Solution

4. Examine the impact of centralized DNS architecture on application performance

In the given scenario, all DNS requests are directed to a **single primary DNS server** , resulting in DNS lookup times of approximately **800 ms**. This centralized architecture creates both a performance bottleneck and a reliability risk.

DNS resolution occurs before a user's browser can establish communication with the web server using the resolved IP address. Therefore, a slow DNS response directly delays application access.

For example, if DNS resolution takes 800 ms and the subsequent TCP connection and HTTP request require additional time, the user experiences a significant delay before the application begins loading.

Major impacts

- 1. Increased application startup time:** Every cache miss must wait for the central DNS server.
- 2. Server overload:** Millions of simultaneous queries can exceed the capacity of the primary DNS server.
- 3. Increased network latency:** Users located far from the DNS server experience longer round-trip times.
- 4. Single point of failure:** If the primary DNS server fails, users may be unable to resolve the application's domain.
- 5. Poor performance during traffic spikes:** Promotional events create sudden increases in DNS queries, making the centralized architecture especially vulnerable.
- 6. Reduced scalability:** Increasing the capacity of one server is less effective than distributing DNS services geographically.

5. Design a distributed DNS solution that minimizes lookup latency while maintaining high

availability

A distributed architecture should replace the single DNS server with multiple geographically distributed DNS nodes.

Proposed architecture

Global Users



Recursive DNS Resolver / Local Cache



Anycast DNS IP



Nearest/Best Available DNS Node



Authoritative DNS Cluster



Application IP Address

Components

1. Multiple authoritative DNS servers

Deploy authoritative DNS servers across different geographic regions. Each location should contain redundant DNS servers.

2. Anycast routing

All DNS nodes can advertise the same Anycast IP addresses. Routing infrastructure directs DNS requests toward an appropriate available node.

3. DNS caching

Recursive resolvers cache DNS responses according to their TTL values. Repeated requests can therefore be answered locally without contacting the authoritative infrastructure.

4. Redundant DNS records

DNS data should be replicated across authoritative servers so that every operational site can provide consistent answers.

5. Health monitoring

Continuous health checks should monitor:

- DNS response time

- Server availability

- Packet loss

- CPU and memory utilization

- Query failure rate

If a DNS location fails, routing should stop directing new traffic toward that location.

6. Load distribution

DNS traffic should be distributed across multiple sites rather than being processed by one central server.

6. Analyze TTL values, Anycast routing, and DNS caching

TTL values

Time To Live (TTL) determines how long a DNS record may remain cached.

Higher TTL

Advantages:

- Fewer DNS queries

- Lower authoritative-server load

- Faster responses from caches

- Better resilience during temporary authoritative-server problems

Limitation:

Changes to DNS records take longer to propagate.

Lower TTL

Advantages:

DNS changes propagate more quickly.

Limitations:

More frequent DNS queries

Higher DNS infrastructure load

Potentially higher latency

Therefore, TTL should be selected according to the stability of the DNS record.

Anycast routing

Anycast allows multiple DNS servers in different locations to advertise the same IP address.

Benefits

Reduces geographic latency

Distributes traffic

Improves scalability

Provides resilience against site failures

If one DNS location fails, routing can direct users toward another operational location.

Limitation

Anycast requires careful routing configuration and monitoring. Incorrect routing can result in traffic being directed to an inefficient location.

DNS caching

Caching is one of the most important mechanisms for reducing DNS latency.

If a resolver already has a valid DNS record in its cache, it can respond immediately without

contacting the authoritative server.

For example:

Without cache:

User → Resolver → Authoritative DNS → Resolver → User

With cache:

User → Resolver → User

Therefore, caching can reduce both latency and DNS-server workload.

Conclusion

The centralized DNS architecture should be replaced with a **distributed Anycast-based DNS infrastructure with redundant authoritative servers, effective caching, appropriate TTL values, and continuous health monitoring** . This design can handle large traffic spikes while maintaining low lookup latency and high availability.

Case Study 4: TCP Latency in an Electronic Stock Trading Platform

1. Analyze three possible TCP-related factors responsible for latency increase

The trading platform experiences an increase in average acknowledgment latency from **1 ms to 40 ms** during market opening hours. The observed TCP retransmissions, longer connection establishment times, and excessive buffering indicate that several TCP mechanisms may be contributing to the problem.

The three important factors are:

1. TCP flow control
2. Nagle's algorithm
3. SYN queue limitations

A. TCP Flow Control

TCP uses a **receive window** to prevent a sender from transmitting more data than the receiver can process.

The receiver advertises its available buffer space using the TCP receive window. If the receiver's application is unable to process incoming data quickly enough, the advertised window may become smaller.

How it affects the trading system

During market opening hours, thousands of orders may be processed simultaneously. If the receiving application or server becomes overloaded, TCP buffers can fill up.

This can result in:

- Reduced transmission rate
- Increased queuing
- Additional buffering
- Delayed acknowledgments
- Increased transaction latency

In extreme cases, a zero-window condition can temporarily prevent additional data from being transmitted.

Effect on high-frequency transactions

Stock trading systems are highly sensitive to milliseconds of delay. If TCP flow control limits the transmission rate, order messages and acknowledgments may wait in buffers instead of being processed immediately.

B. Nagle's Algorithm

Nagle's algorithm is designed to reduce the number of small TCP packets transmitted over a network.

When an application generates a small amount of data, Nagle's algorithm may temporarily

hold that data and combine it with additional data before transmission.

This is useful for reducing network overhead, but it can be harmful to latency-sensitive applications.

Example

Suppose a trading application sends a very small order message. Instead of immediately transmitting the packet, TCP may wait for an acknowledgment or additional data before sending the next small segment.

This waiting period can increase transaction latency.

Effect on high-frequency trading

Electronic trading systems frequently exchange small messages such as:

- Buy orders

- Sell orders

- Order modifications

- Acknowledgments

- Trade confirmations

These messages require extremely low latency. Therefore, waiting to combine small packets can be undesirable.

For latency-sensitive TCP connections, disabling Nagle's algorithm using **TCP_NODELAY** may reduce unnecessary delays.

However, this should be tested carefully because disabling Nagle can increase the number of small packets and network overhead.

C. SYN Queue Limitations

The TCP connection establishment process begins with the **three-way handshake** :

Client → SYN → Server

Server → SYN-ACK → Client

Client → ACK → Server

During periods of very high connection attempts, the server's TCP SYN queue can become saturated.

If the queue is too small, new connection requests may be delayed or dropped.

Effect on the trading system

During market opening hours, many users, trading applications, or connections may become active simultaneously. If the SYN backlog is insufficient, new connections may experience:

Connection delays

Retransmitted SYN packets

Failed connection attempts

Increased connection establishment time

This contributes directly to the observed increase in latency.

2. Explain how each factor affects transaction processing in high-frequency systems

TCP factor	Mechanism	Effect on trading
Flow control	Limits sending based on receiver capacity	Queuing and delayed order processing
Nagle's algorithm	Combines small TCP segments	Additional delay for small messages
SYN queue limitations	Limits pending connection requests	Slow or failed connection establishment
TCP retransmissions	Resends lost packets	Additional latency

Excessive buffering Holds packets in queues Increased end-to-end delay

Flow control and transaction processing

Trading systems generate large numbers of small transactions within very short time intervals. If the receiving application cannot process these messages rapidly enough, TCP flow control reduces the sender's transmission rate.

This creates a chain reaction:

High order rate → Full receive buffer → Smaller TCP window → Reduced transmission → Queuing → Increased latency

Nagle's algorithm and transaction processing

Because trading messages are often small, Nagle's algorithm can introduce additional waiting time. Even a few milliseconds can be significant when the normal acknowledgment latency is approximately 1 ms.

SYN queue and transaction processing

If new connections are frequently established during peak activity, a limited SYN queue can cause connection establishment delays. A trading platform may therefore spend additional time establishing connections instead of processing transactions.

For extremely latency-sensitive systems, maintaining persistent connections can reduce the need for repeated TCP handshakes.

3. Recommend appropriate TCP configuration changes

Several TCP and application-level improvements can be implemented.

1. Disable Nagle's algorithm for latency-sensitive connections

Enable **TCP_NODELAY** where appropriate.

This allows small messages to be transmitted immediately rather than waiting to combine

them with additional data.

Benefit: Lower latency for small order and acknowledgment messages.

Consideration: It can increase packet overhead, so it should be applied selectively.

2. Increase TCP SYN backlog

The server's SYN backlog should be configured to handle the expected number of simultaneous connection attempts.

Operating-system parameters controlling the TCP listen/SYN queues should be tuned according to the platform's workload.

Benefit: Reduces connection establishment delays and dropped connection attempts during traffic spikes.

3. Use persistent TCP connections

Rather than creating a new TCP connection for every transaction, trading applications should maintain persistent connections where the protocol and security architecture allow it.

This avoids repeated:

SYN → SYN-ACK → ACK

handshakes.

Benefit: Reduces connection establishment overhead and improves transaction latency.

4. Optimize TCP receive and send buffers

TCP buffer sizes should be large enough to support peak traffic but not so large that excessive data remains queued.

The goal should be to balance:

Throughput ↔ Buffering ↔ Latency

Oversized buffers can contribute to queueing delays, while undersized buffers can limit throughput.

5. Reduce packet loss and retransmissions

The observed TCP retransmissions indicate that packet loss or congestion is occurring.

The organization should investigate:

- Network congestion

- Interface errors

- Packet drops

- Faulty network equipment

- Insufficient bandwidth

- Queue congestion

Reducing packet loss is essential because TCP retransmissions directly increase transaction latency.

6. Implement Quality of Service

Trading traffic should receive appropriate priority over non-critical traffic.

For example, network resources used by order messages and confirmations should not be heavily affected by:

- Backups

- Software downloads

- Large file transfers

- Video streaming

- Other non-critical traffic

7. Use connection pooling or persistent sessions

If the trading application requires many logical transactions, maintaining a controlled pool of established connections can reduce the frequency of TCP handshakes.

This can be particularly useful during market-opening traffic spikes.

8. Monitor TCP performance continuously

The organization should monitor:

- TCP retransmission rate

- RTT

- SYN backlog utilization

- Receive-window size

- Send-window size

- Packet loss

- Connection establishment time

- Application queue length

- CPU utilization

Monitoring helps determine whether the primary bottleneck is the network, operating system, TCP stack, or application.

Overall Conclusion

The four case studies demonstrate that **transport protocols and network-layer services have a major influence on application performance** .

In the video-conferencing scenario, real-time multimedia requires low latency and low jitter, making UDP-based real-time transport and application-level mechanisms such as adaptive bitrate, jitter buffering, and FEC appropriate.

For the DNS scenarios, centralized DNS creates high latency, scalability problems, and a single point of failure. A **distributed Anycast DNS architecture combined with caching, DNS pre-fetching, optimized TTL values, redundancy, and health monitoring** can significantly improve performance and availability.

In the stock-trading scenario, TCP flow control, Nagle's algorithm, SYN queue limitations, retransmissions, and excessive buffering can increase transaction latency. Appropriate configuration of **TCP_NODELAY, SYN queues, buffer sizes, persistent connections, QoS, and network capacity** can help reduce latency while maintaining reliable communication.

Overall, the key principle is to select and configure networking mechanisms according to the **specific requirements of the application**. Real-time conferencing prioritizes timely delivery, DNS prioritizes fast resolution and availability, while financial trading prioritizes extremely low latency and reliable transaction processing.

